

Software Architecture Document

Документ архитектуры ПО

1. Introduction (Введение)

Документ описывает архитектуру SCMS с использованием подхода «4+1» представлений (Use-Case, Logical, Process, Deployment, Implementation). Архитектура построена на принципах микросервисной декомпозиции с изоляцией критических данных DHF.

1.1 Purpose (Назначение)

Зафиксировать архитектурные решения и предоставить единое понимание структуры системы всем участникам разработки. Документ используется архитектором, разработчиками, DevOps и QA.

1.2 Scope (Область применения)

Документ относится к проекту SCMS — корпоративной системе управления клиникой sleeving. Описывает архитектуру backend (Java), frontend (React) и взаимодействие с внешними системами.

1.3 Definitions, Acronyms and Abbreviations (Определения)

Определения терминов приведены в документе «Glossary (Глоссарий)» проекта SCMS.

1.4 References (Ссылки)

- SRS (Спецификация требований) проекта SCMS, версия 1.0.
- Vision (Концепция) проекта SCMS, версия 1.0.
- Use Case Specification проекта SCMS.
- Richards M., “Software Architecture Patterns” (O’Reilly, 2015).

1.5 Overview (Обзор)

Раздел 2 — общее представление архитектуры. Раздел 3 — цели и ограничения. Раздел 4 — Use-Case View (прецеденты). Раздел 5 — Logical View (модули). Раздел 6 — Process View (процессы). Раздел 7 — Deployment View (развёртывание). Раздел 8 — Implementation View (структура кода). Раздел 9 — характеристики производительности. Раздел 10 — качество архитектуры.

2. Architectural Representation (Представление архитектуры)

Система построена по клиент-серверной архитектуре с микросервисным бэкендом. Frontend разделён на два веб-приложения: внешний портал (Meth) и внутренний портал (персонал). Коммуникация между сервисами осуществляется через message queue (RabbitMQ).

Diagram/View	Use Case	Logical	Implementation	Process	Deployment
Use Case Diagram	+	-	-	-	-
Class Diagram	+	+	+	-	-
Activity Diagram	+	+	-	-	-
Sequence Diagram	-	+	-	+	-

Component Diagram	-	-	+	-	-
Package Diagram	-	-	+	-	-
Deployment Diagram	-	-	-	-	+

3. Architectural Goals and Constraints (Цели и ограничения)

Архитектурно-значимые факторы:

1. Безопасность данных DHF: Все данные стеков шифруются квантовым алгоритмом. Audit-log неизменяем и доступен только администратору.
2. Изоляция сети: Внутренний портал работает только в изолированной сети клиники (intranet). Внешний портал доступен через защищённый HTTPS.
3. Отказоустойчивость: Процедура needlecaster не должна прерываться. Система должна обеспечивать 100% доступность во время активных процедур. Используется message queue для буферизации операций.
4. Аудируемость: Каждая операция со стеком фиксируется в неизменяемом audit-log. Журнал доступен только администратору.
5. Масштабируемость: Микросервисная архитектура с горизонтальным масштабированием. Цель: поддержка до 50 одновременных пользователей.
6. Стерильный интерфейс: Needlecaster управляет системой жестами/голосом без прямого касания. Распознавание локальное, без передачи данных во внешние сети.

4. Use-Case View (Прецеденты)

Прецеденты системы описаны в спецификации SRS (раздел 2.1) и детализированы в Use Case Specification. Ключевые архитектурно-значимые прецеденты:

- UC-03 Резервирование sleeve — определяет требования к согласованности данных (блокировка ресурса, авто-снятие резерва).
- UC-05 Мониторинг needlecaster — определяет требования к real-time обновлению данных (WebSocket, задержка ≤ 100 мс).
- UC-06 72-часовая валидация — определяет требования к таймерам, чекпоинтам и автоматической генерации PDF.

5. Logical View (Логическое представление)

5.1 Overview (Обзор)

Система разделена на 4 логических слоя: Presentation (React), API Gateway (Spring Cloud Gateway), Business Services (микросервисы), Data Layer (PostgreSQL + RabbitMQ).

5.2 Architecturally Significant Design Packages (Значимые пакеты)

Backend (Java, Spring Boot):

- sleeve-service — учёт инвентаря, культивирование, резервирование, мед. карты.
- stack-service — реестр стеков, история перемещений, audit-log.
- procedure-service — мониторинг needlecaster, тестирование, таймеры.
- validation-service — 72-часовая валидация, чекпоинты, сертификация (PDF).
- user-service — аутентификация, RBAC, управление пользователями.
- client-service — онбординг, контракты, документы, профили Meth.
- notification-service — отправка уведомлений (email/SMS/bell icon).

- reporting-service — генерация отчётов, поиск, экспорт PDF/Excel.
- api-gateway — единая точка входа, маршрутизация, rate limiting.

Frontend (React):

- mfe-client-portal — внешний портал для Meth (минималистичный UI).
- mfe-staff-portal — внутренний портал для персонала (три интерфейса: Broker, Needlecaster, Psychosurgeon, Admin).
- shared-ui-kit — общие компоненты (таблицы, формы, дашборды, индикаторы статусов).

6. Process View (Процессное представление)

Потоки выполнения:

1. Пользовательские запросы (синхронные): REST API (Spring MVC) — поиск, просмотр, редактирование данных. Таймаут: 5 секунд.
2. Мониторинг needlecast (асинхронный): WebSocket (STOMP) + RabbitMQ. Сервис procedure-service публикует обновления каждые 100 мс. Frontend подписывается через WebSocket.
3. Автоматические процессы (фоновые задачи):
 - Auto-release reservation: Scheduled task, каждые 60 минут проверяет просроченные резервирования.
 - Notification dispatch: RabbitMQ consumer, обрабатывает очередь уведомлений.
 - Certificate generation: Асинхронная генерация PDF после успешной валидации.
4. Аудит-лог (асинхронная запись): Все сервисы отправляют события в RabbitMQ → consumer записывает в таблицу audit_log. Запись синхронно дублируется в отдельный файл на диске (неизменяемость).

7. Deployment View (Развёртывание)

Система развёртывается на одном сервере (учебный сервер helios.cs.ifmo.ru FreeBSD 14.4-STABLE) с использованием Docker Compose. Каждый микросервис — отдельный контейнер. PostgreSQL — отдельный контейнер. RabbitMQ — отдельный контейнер. Nginx — reverse проху, терминация TLS.

Сетевые интерфейсы:

- Port 443 (HTTPS) — внешний портал для Meth (доступ из интернета).
- Port 8443 (HTTPS) — внутренний портал (доступ только из intranet клиники).
- Port 5672 (AMQP) — RabbitMQ (только внутренняя сеть Docker).

8. Implementation View (Реализация)

Структура репозитория:

```
scms/
├── backend/
│   ├── api-gateway/           # Spring Cloud Gateway
│   ├── sleeve-service/        # Микросервис sleeves
│   ├── stack-service/         # Микросервис стеков
│   ├── procedure-service/     # Микросервис процедур
│   ├── validation-service/    # Микросервис валидации
│   ├── user-service/          # Микросервис пользователей
│   ├── client-service/        # Микросервис клиентов
│   ├── notification-service/  # Микросервис уведомлений
│   ├── reporting-service/     # Микросервис отчётов
│   └── shared/                # Общие библиотеки (DTO, утилиты)
└── frontend/
```

		mfe-client-portal/	# Микрофронтенд Meth
		mfe-staff-portal/	# Микрофронтенд персонала
		shared-ui-kit/	# Общие UI-компоненты
		infra/	
		docker-compose.yml	# Оркестрация
		nginx/	# Конфигурация Nginx
		db/	# Миграции Flyway
		docs/	# Документация

9. Size and Performance (Размер и производительность)

- Среднее время отклика API: ≤ 300 мс (95 перцентиль).
- Задержка WebSocket (мониторинг needlecaster): ≤ 100 мс.
- Генерация PDF сертификата: ≤ 5 секунд.
- Поиск в каталоге sleeves (50 000 записей): ≤ 2 секунды.
- Максимальное количество одновременных пользователей: 150 сессий.
- Потребление памяти (пик): ≤ 16 ГБ.
- Размер БД (архив 1 год): 500 ГБ.

10. Quality (Качество)

Архитектура удовлетворяет следующим атрибутам качества:

- Надёжность: Каждый микросервис stateless; при отказе — автоматический перезапуск Docker. Данные DHF дублируются в реальном времени. RabbitMQ обеспечивает гарантированную доставку сообщений.
- Безопасность: Квантовое шифрование данных стеков (симметричный ключ + смена каждые 5 минут). 2FA для Meth. RBAC на уровне API Gateway. Audit-log с однократной записью (append-only).
- Масштабируемость: Микросервисы масштабируются горизонтально (docker-compose up – scale). API Gateway — единая точка входа с балансировкой.
- Модифицируемость: Микросервисная архитектура позволяет изменять/добавлять модули без влияния на остальные. Message queue (RabbitMQ) обеспечивает слабую связанность сервисов.